Lab Assignment 8: SHIFT-REDUCE PARSING OF AN AMBIGUOUS GRAMMAR

Course: CS3104 - Compiler Design Laboratory

Syllabus Unit: Shift-Reduce Parsing with Precedence and Associativity

Date: 08/10/2026

Instructions

- Write your name and roll number on top of the source file as comments.
- Source file must be named exactly: shift_reduce.c
- Each program must compile cleanly with plain gcc, e.g.: gcc shift_reduce.c -o shift_reduce (no parser-generator tools such as yacc/bison are to be used — the point of this lab is to build the parsing logic by hand).
- Submit only the three source files through the same submission portal used for previous assignments – http://10.22.11.15/~abyaym/2026/CS3104_26/submission/

Problem Statement

A parser sits above the lexical analyzer and decides whether a stream of tokens conforms to a context-free grammar. In bottom-up shift-reduce parsing, the parser shifts input symbols onto a stack and reduces recognized right-hand sides to their corresponding non-terminals. Ambiguous expression grammars can create shift/reduce conflicts because more than one valid action may be possible at a parsing step. In this assignment, implement a simple shift-reduce parser and resolve such conflicts using explicit operator precedence and associativity. The parser must recognize expressions generated by the following ambiguous grammar and use the given rules to decide when to shift and when to reduce.

You will work with two versions of the language of parenthesized **id / + / *** expressions:

```
Ambiguous grammar G1:

Production Rule :  1 E → E + E

                   2 E → E - E

                   3 E → E * E

                   4 E → id
```

Example

Input string (tokens already classified by your Lab 3 scanner):

```
id + id * id
```

Expected output (excerpt, one section per program):

```
SHIFT: id

REDUCE: id -> E

SHIFT: + SHIFT: id

REDUCE: id -> E

SHIFT: * SHIFT: id

REDUCE: id -> E
```

```
REDUCE: E * E -> E

REDUCE: E + E -> E

ACCEPT
```

Core Requirements

- Represent the parser state using a stack and an input stream.
- Recognize id as an operand and reduce id → E after shifting it.
- When the stack contains E operator E and another operator is the lookahead, detect the possible shift/reduce conflict.
- If the stack operator has higher precedence than the incoming operator, REDUCE.
- If the incoming operator has higher precedence, SHIFT.
- If the two operators have equal precedence, use left associativity and REDUCE.
- Print each SHIFT and REDUCE operation, and explicitly print the conflict-resolution decision where applicable.
- After all input is processed, reduce remaining valid handles and print ACCEPT only when the stack is reduced to $ E; otherwise print REJECT

Programming Task

Write a C program named shift_reduce.c that reads an expression containing id, +, -, and *, performs shift-reduce parsing, uses the specified precedence/associativity rules to resolve conflicts, prints each SHIFT/REDUCE decision, and displays the final ACCEPT/REJECT result.

Implementation Guidelines

- Input Handling Read an arithmetic expression containing:
    - id , +, *, (,),

  Add $ at the end of the input to indicate the end of input.
- **Stack Implementation** : Maintain a stack containing grammar symbols. Initially, the stack should contain $.

  Use a suitable C data structure.
- **REDUCE Operation**: Check whether the top of the stack matches the right-hand side of a grammar production; if matched, replace the RHS with E.
- **Ambiguity and Conflicts**: Detect situations where both SHIFT and REDUCE operations are possible and identify them as SHIFT/REDUCE conflicts.
- **Conflict Resolution**: Resolve conflicts using operator precedence and associativity, where * has higher precedence than +, and both + and * are left-associative.
- **Parentheses**: Give parentheses the highest priority and reduce the expression inside parentheses before applying the surrounding arithmetic operators.

- ACCEPT and REJECT Conditions: Accept when the stack contains $ E and the input is $; if no valid SHIFT or REDUCE operation is possible, reject the expression.

- **Compilation & Testing**

```
gcc shift_reduce.c -o shift_reduce
./shift_reduce < input.txt
```

Deliverables

- shift_reduce.c